

FR5 ACT (DINOv2 / DINOv3) — Failure Analysis & Fixes

FR5 ACT (DINOv2) — Failure Analysis

1. Symptoms and Evidence — what the robot actually did
2. The Failure Modes Explained (from scratch)
3. Training-Log Findings — what the training tells us (and the bug it hides)
4. Root Cause and Fixes — what to actually do
5. Glossary and References
6. The CVAE KL Term — a from-scratch deep dive
7. Grasp-Success Gate — requirements (deferred, needs a sensor signal)
8. The Fix — what we actually changed (branch `fix/dino-act-cvae-probe`)
9. “Is the model too small?” — No. It’s overfitting, not under-capacity.
10. Free-bits, explained in detail (the “tax-free allowance” for the latent)

FR5 ACT (DINOv2) — Failure Analysis

This folder is a complete, from-the-ground-up explanation of **why the trained ACT + DINOv2 policy failed on the real FR5 robot**, and what to do about it. It is written so that you can hand it to anyone on the team and they will understand the problem fully — no prior context needed.

The one-sentence answer

The policy learned to reproduce the **average demonstrated trajectory** and to **ignore the cameras**, so on the robot it reaches toward the *average* marker location, **freezes** near the grasp, **closes the gripper in mid-air**, and — because it never actually grasps anything — **never starts the move-to-bowl phase**.

That single sentence is the product of **three well-known imitation-learning failure modes stacking on top of each other**, plus **one concrete training bug** (the CVAE turned itself off) that made the whole thing worse. Each is documented in detail in the files below.

What was analysed

- **The model:** lerobot ACTPolicy (CVAE + transformer, chunk 100, temporal ensembling 0.01) with a **frozen DINOv2 ViT-S/14** vision backbone, **two cameras** (wrist + scene), 7-D action (6 joints + gripper). Some runs used **proprioception dropout 0.5** and **gripper dropout 0.5**. This is the `act` branch of `vivek-kanjarla/fairino-fr5-policies`.
- **The data:** training logs (5 ACT runs + 1 Octo run) and inference/deploy logs (12 robot rollouts + 2 diagnostic `.npz` files) that you uploaded — see `experiment_data/`.
- **The task:** pick up a marker and place it in a bowl.

How to read this folder (in order)

#	File	What it covers
1	01_symptoms_and_evidence.md	Exactly what the robot did, mapped to the hard numbers from your deploy logs and diagnostic files. The “crime scene.”
2	02_failure_modes_explained.md	Every named phenomenon explained from scratch — idle-action freeze, copycat/inertia, mode averaging, proprioceptive shortcut, gripper-state ambiguity, covariate shift — with the mechanism, the citation, and how it produced <i>your</i> symptom. The deep dive.
3	03_training_log_findings.md	What the training logs reveal: the CVAE posterior collapse (the big bug), the val-loss anti-correlation, the loss plateau, corrupted frames, and a per-run comparison.
4	04_root_cause_and_fixes.md	The full causal chain, the root causes ranked, and a prioritized, actionable fix list with how to validate each one. Start here when you want to <i>do</i> something.
5	05_glossary_and_references.md	Plain-language definitions of every term, and all the papers with links.
6	06_cvae_kl_deep_dive.md	The CVAE KL term explained from scratch — what KL divergence is, the ELBO, the reparameterisation trick, and exactly why <code>kl_weight=10</code> caused posterior collapse (the top training bug). Read after file 3.
7	07_grasp_gate_requirements.md	The deferred grasp-success gate — what sensor signal it needs (the FR5 gripper is command-only) and where it plugs into <code>deploy.py</code> .
8	08_the_fix_implementation.md	What we actually changed (branch <code>fix/dino-act-cvae-probe</code>) — the CVAE fix (free-bits + annealing + lower weight,

#	File	What it covers
		explained), the DINOv2 2-cam port, the vision-vs-state probe, the full change set, and what's verified vs left.
9	09_model_capacity_vs_overfitting.md	“Is the model too small?” — No. Why the large train/val gap is overfitting (data-bound), not under-capacity; the 32-dim latent is standard; scale <i>data</i> not params.
10	10_free_bits_explained.md	Free-bits explained in detail — the “tax-free allowance per latent dim” idea, the gradient view (ASCII curve), a worked example, the code, and why $\lambda=0.03$ is kept gentle.

The 30-second version (if you read nothing else)

WHAT YOU SAW LOGS	WHAT IT IS CALLED	PROOF IN YOUR
reaches ~same spot every time → proprio 2.2x > vision ignores where the marker is	proprioceptive shortcut / causal confusion (de Haan'19)	→ your ablation:
grasps a "mean" location, collapsed to ~0 in training object-blind, in mid-air plain regression => mean	→ mode averaging / regression to the mean (Diffusion Policy)	→ CVAE KL => ACT became
freezes near the marker 0.0 mm motion for 436 steps; current pose (0.84 deg)	→ idle-action problem (Chi et al. 2023) + copycat	→ deploy_grip2: chunk0 ≈
closes gripper on nothing, at 360–403 mm from start, then "retracts to transport" retracts as if carrying it	→ gripper-state ambiguity (arXiv 2503.23835)	→ gripper CLOSED then arm
never reaches the bowl phantom grasp it is out of no recovery behaviour	→ compounding error / covariate shift (Ross 2011)	→ after the distribution;

The most important single fix: the CVAE latent collapsed during training ($KL \rightarrow \sim 0$), which silently turned ACT into ordinary behavioral cloning — and ordinary BC mode-averages. Fix that first (see file 4), then fix the data (fixed home + varied marker) and filter idle frames. Details and the full priority list are in [04_root_cause_and_fixes.md](#).

Note: the analysis is evidence-based — every claim in these files is tied either to a specific number in your logs (`experiment_data/...`) or to a named paper. Where something is inferred rather than directly measured, it says so.

1. Symptoms and Evidence — what the robot actually did

This file is the **crime scene**: the precise, measured behaviour of the policy on the real FR5, pulled directly from your deploy logs (`experiment_data/inference_logs/`) and the two diagnostic NumPy files. No interpretation yet — just *what happened* and *the numbers that prove it*. The explanations of *why* are in [02_failure_modes_explained.md](#) .

1.1 The reported behaviour (operator's words, made precise)

“Almost every time, the robot reaches near the marker, then freezes, then goes to the end state. It does not pick the marker — or if it does, it grasps mid-air and stops. We never saw it move toward the bowl.”

Broken into four distinct, separately-measurable events:

1. **Approach** — the arm leaves home and sweeps toward roughly where the marker usually is.
2. **Freeze** — partway there (near the grasp), motion collapses; the arm essentially stops.
3. **Phantom grasp** — the gripper closes while the hand is still in the air (nothing in it).
4. **No transport** — the arm either stays frozen or retracts a little, but the “carry the marker to the bowl” phase never happens.

Every one of these is visible in the logs. Here is the evidence, event by event.

1.2 Event 2 — the FREEZE (this is the most clear-cut)

What “freeze” means numerically: the commanded joint angles stop changing, and the end-effector (EEF / TCP) position stops moving, for hundreds of control steps.

Direct evidence

Log file	What it shows
deploy_grip2.out	The arm moves slightly from home, then EEF displacement = [0.0, 0.0, 0.0] mm from step 134 through step 570 — i.e. 436 consecutive control steps with zero motion . The frozen pose is only 1.48° (mean) from the home pose .
deploy_bowl.out	After step ~663 the TCP moves < 0.6 mm over 1329 steps (402.6 → 403.2 mm). Joint motion in the frozen phase is ~0.021°/step vs ~0.056°/step while still active — but the <i>tool tip</i> is completely still.
deploy_prop.out	Reaches ~393.9 mm EEF by step ~472, then joint positions change < 5° total over the next 1517 steps .
intervention.out	Under policy control the arm only ever moved 2–4 mm from rest before stalling; a human took over and moved 129 mm in the same setup in under 2 minutes. The policy is ~30–60× weaker at making progress than a human in the identical scene .

The offline fingerprint of the freeze

From `diag_offline.npz` (90 inference calls on *training-distribution* observations):

- `mean |chunk0 - current_state| = **0.84°**` — the **very first action** the policy predicts is, on average, **less than one degree** away from where the arm already is. The policy’s default prediction is “*stay almost exactly where you are.*”
- The full 100-step chunk only spans `mean |chunk[99] - chunk[0]| = 5.75°` per joint — it *plans* small motions, with **tiny first steps**.

Plain-language reading: the model has learned that the correct next action is “barely move.” That is fine while it is still approaching (lots of barely-moves add up), but near the grasp the barely-moves average out to **nothing**, and the arm stops. This is the signature of the **idle-action problem** (file 2, §2.1).

1.3 Event 3 — the PHANTOM (mid-air) GRASP

What it means numerically: the gripper command crosses its “close” threshold while the end-effector is still far from the marker / table — i.e. the hand closes on empty space.

Direct evidence

Log file	Gripper close event	EEF distance from start at that moment
deploy_ft.out	[GRIPPER] CLOSED at step ~553	386.9 mm — then the arm retracts to 312.6 mm (as if carrying something)
deploy_grip4.out	[GRIPPER] CLOSED at step ~526	363.6 mm — then retracts to 285.1 mm
deploy_bowl.out	gripper jumps 0.02 → 0.48 at step 663	402.6 mm , no contact; then oscillates 0.43–0.63 for 1329 steps (never commits)
deploy_dino.out / deploy_dino2.out	[GRIPPER] CLOSED at step 2	27 mm / 0 mm — closes <i>immediately</i> , before any reach. (These runs had no gripper dropout in training.)

Two distinct sub-patterns:

- **Without gripper dropout** (`deploy_dino*`): the gripper closes at **step 2**, basically at the start pose. The model learned “always start closed” — it never even tries to localise.
- **With gripper dropout** (`deploy_ft` , `deploy_grip4` , `deploy_bowl`): the gripper closes **mid-reach** (360–403 mm out), then the arm **switches into a “transport” motion** (it retracts) — behaving exactly as if it had successfully grabbed the marker, even though it grabbed air.

Plain-language reading: the gripper close is fired by **trajectory timing / arm pose**, not by actual contact with the object. And because the policy’s only notion of “am I holding something?” is the binary gripper state (closed = “I have it”), once it closes it *believes* it succeeded and moves to the next phase. This is **gripper-state ambiguity** (file 2, §2.4).

1.4 Event 1 + 4 — APPROACH to a fixed spot, and NO bowl

What it means: the arm reaches toward the *same* region every rollout, regardless of where the marker actually is, and never proceeds to the place phase.

Direct evidence

- The frozen/grasp poses across `deploy_bowl` , `deploy_prop` , `deploy_ft` , `deploy_grip4` land in a **similar EEF band (~360–403 mm out)** — a *fixed average reach*, not one that tracks the marker.
- **Your own causal-intervention ablation** (the reason the `_prop` config exists, quoted in `config.joint_dino_prop.yaml`): “*proprio influenced the action 2.2× more than vision.*” The policy’s output depends **2.2× more on the joint state than on the cameras** — so it reaches the same place no matter what the camera sees.

- In **every** log, after the phantom grasp the EEF magnitude **never decreases toward a different (bowl) location**; the task simply ends in a freeze or a small retract.

Plain-language reading: the policy is **object-blind**. It reproduces the average reach and the average grasp timing, ignoring the actual marker position — the **proprioceptive shortcut / mode averaging** (file 2, §2.2 and §2.3). And since the grasp never really succeeds, the later “move to bowl” behaviour — which in the data *a/ways* begins from “object in hand” — is never triggered (**covariate shift**, file 2, §2.5).

1.5 The diagnostic .npz files (the quantitative core)

These two files were produced by the policy’s own instrumentation (`predict_with_diag` / `predict_chunk` on the `act` branch). They are the most precise evidence.

`diag_offline.npz` — 90 open-loop predictions on in-distribution data

Array	Shape	Meaning
<code>chunks</code>	(90, 100, 7)	full predicted 100-step action chunks
<code>chunk0</code>	(90, 7)	first action of each chunk
<code>ensembled</code>	(90, 7)	temporally-ensembled output (what gets executed)
<code>state</code>	(90, 7)	actual joint state at prediction time
<code>latency_ms</code>	(90,)	mean 13.1 ms , max 137 ms (well under the 33 ms budget → timing is NOT the problem)

Key numbers:

- $\text{mean } |\text{chunk0} - \text{state}| = **0.84°** \rightarrow \text{first action} \approx \text{“stay put” (the freeze fingerprint)}.$
- $\text{mean } |\text{ensembled} - \text{chunk0}| = **4.87°** \text{ (max } 16.4°) \rightarrow \text{temporal ensembling is adding motion, not cancelling it (it blends older chunks' future steps)}.$
- Direction agreement between `chunk0` and `ensembled` $\approx$ **58.7%** → barely above chance: the successive predictions point in **incoherent directions** ~41% of the time.

diag_receding.npz — 346 steps of pure receding-horizon (temporal ensembling OFF)

Finding	Number	Meaning
Robot tracking error	<code>mean executed - actual joints = **0.26**</code>	the hardware tracks commands accurately — not a robot problem
Per-step motion	<code>mean Δjoint = **0.17°/step**</code>	without ensembling the arm barely moves
Sign-flip rate	68%	each fresh <code>chunk0</code> predicts the opposite direction from the previous one 68% of the time → near-pure oscillation/jitter
Total joint range over 345 steps	J1 7.2°, J4 13.6°, others < 5°	the arm essentially never leaves home without ensembling

Important nuance about temporal ensembling: turning it **off** made things *worse* (`deploy_receding.out`: the arm oscillated within 33 mm and never progressed). So temporal ensembling is **not** the villain — it is actually the only thing letting the arm travel at all. The 68% sign-flip rate shows the *underlying predictions* are incoherent; ensembling smooths them into forward motion, but it cannot fix a policy that fundamentally wants to stop. (This rules out “temporal-ensembling cancellation” as the root cause.)

1.6 Offline looks OK, online is broken (the covariate-shift tell)

A crucial detail: **all the checkpoints have similar, reasonable validation loss** (`val_l1` $\approx$ 0.144–0.174 — see file 3), and the **offline** predictions in `diag_offline.npz` look like sane small motions. Yet **online** the rollouts consistently fail.

That gap — *offline metrics fine, closed-loop rollout broken* — is the classic signature of **compounding error / covariate shift** (file 2, §2.5): the policy is only ever tested, offline, on states the experts visited; online it drifts into states it never saw and has no good action for, so it regresses to “do almost nothing.”

1.7 Things that are NOT the problem (ruled out by the data)

Hypothesis	Verdict	Evidence
Robot hardware / tracking	Not it	tracking error 0.26°; commands are followed accurately
Control-rate / latency	Not it	inference 12–13 ms, far under the 33 ms (30 Hz) budget
Numerical instability (NaN/Inf)	Not it	no NaNs/Infs in any log or array
Temporal-ensembling cancellation	Not the root cause	turning it off made motion <i>worse</i> (33 mm jitter)
A single bad checkpoint	Not it	the failure reproduces across epochs 19/28/40/53 and across configs
<code>deploy_receding.out</code> crash	Setup issue, not policy	<code>OSError:</code> <code>GetActualJointPosDegree</code> returned <code>14</code> = CNDE disconnect from a suspended <code>teleop.py</code> sharing the port

1.8 Evidence → phenomenon map (forward reference)

Measured fact	Points to	Detailed in
<code>chunk0</code> $\approx$ state (0.84°); 436-step 0 mm freeze	Idle-action problem	file 2 §2.1
68% sign-flip; near-zero net motion	Copycat / inertia (same family)	file 2 §2.1
proprio 2.2× > vision; same reach regardless of marker	Proprioceptive shortcut / causal confusion	file 2 §2.2
object-blind “average” grasp; CVAE off in training	Mode averaging / regression to mean	file 2 §2.3
gripper closes at 360–403 mm, then “transports” nothing	Gripper-state ambiguity	file 2 §2.4
offline OK, online broken; no recovery after phantom grasp	Covariate shift / compounding error	file 2 §2.5

Next: [02_failure_modes_explained.md](#) explains each of these from first principles.

2. The Failure Modes Explained (from scratch)

This is the deep dive. Each section below is **one named phenomenon** from the imitation-learning literature. For each you get: a plain-language explanation, the precise mechanism (with the small amount of math that matters), the **canonical paper(s)**, and exactly **how it produced your symptom** with the evidence from [file 1](#).

Read the preamble first — it explains *why behavioral cloning is fragile in the first place*, which is the common root of everything that follows.

2.0 Preamble — why behavioral cloning breaks at all

Your ACT policy is trained by **behavioral cloning (BC)**: you record human demonstrations as (observation, action) pairs and train a network to output the action the human took. The objective is just supervised learning:

```
minimize E[ (π(observation) - action)2 ]
```

That's it. There is **no notion of the goal, no feedback loop, and no understanding of cause and effect** — only “given this input, copy that output.” Three structural gaps follow, and every failure mode below lives in one of them:

Gap	Consequence	Failure modes
BC fits correlations , not causes	it can lean on inputs that merely correlate with the action (joint state, previous action) instead of the ones that <i>cause</i> it (the camera)	§2.1 copycat, §2.2 proprioceptive shortcut
BC minimises average error → predicts the mean	when several actions are valid it outputs their average, which can be invalid	§2.3 mode averaging
BC is only trained on expert states	at deploy it drifts into states it never saw and has no good action	§2.5 covariate shift

ACT tries to patch the second gap with a **CVAE** (a latent “style” variable that lets it pick *one* mode instead of averaging). **In your runs that patch failed** — the CVAE collapsed (see [file 3](#)) — so the mean-averaging behaviour came back in full.

2.1 The idle-action problem (and its cousin, the copycat/inertia problem)

This is the FREEZE. The single most-supported cause in your data.

Plain language

Humans pause while teleoperating — they slow down to line up the grasp, they hesitate, they hold still for a moment before closing the gripper. Those pauses get recorded as **frames where the action is “stay still.”** Because BC weights every frame equally, and “stay still” frames cluster at the **pre-grasp pose**, the network learns a very confident rule: “*when you are near the grasp pose, do nothing.*” At deploy, the arm reaches that pose and the policy votes, confidently, for **no motion** — it freezes.

The mechanism, precisely

Two reinforcing effects:

1. **Idle frames are low-loss and over-represented.** Predicting “stay still” at a state where the demo was still has near-zero error, and there are many such frames, so the gradient pushes the policy hard toward “stop here.” (Chi et al., the Diffusion Policy paper, name this the “**idle actions**” problem and report a single-step BC baseline got *stuck in 40% of real-world trials* until idle frames were explicitly removed.)
2. **The copycat / inertia shortcut.** Consecutive expert actions are almost identical (the arm barely moves between two 30 Hz frames). So the *easiest* function for the network to learn is “output $\approx$ the previous action.” If the previous action was “slowing down,” the policy keeps outputting “slowing down” — a self-perpetuating stop. Wen et al. (NeurIPS 2020) name this the “**copycat problem**” and prove it happens when (a) actions are temporally correlated and
 - b. the previous action is recoverable from the input — both true here.

There is a subtle but important interaction with your **absolute-joint action space**: the policy predicts *absolute* target joint angles, not deltas. “Do nothing” in absolute space means “predict my current joint angles” — which, on average, is exactly what the data shows near the pause. So the absolute action space makes “predict $\approx$ current pose” the lowest-loss answer.

How it produced YOUR symptom

- `diag_offline.npz`: `mean |chunk0 - current_state| = **0.84°**` — the policy’s default first action is “essentially stay where you are.” That is the idle-action rule, measured directly.
- `deploy_grip2.out`: **0.0 mm motion for 436 steps** near home — the arm reached a “stay still” basin and never left.
- The absolute-joint design (above) is *why* “stay” is so cheap to predict.

Canonical papers

- Chi et al., *Diffusion Policy* (RSS 2023) — names “idle actions,” prescribes removing them. [arXiv:2303.04137](#)
 - Wen et al., *Fighting Copycat Agents in Behavioral Cloning from Observation Histories* (NeurIPS 2020). [arXiv:2010.14876](#)
-

2.2 The proprioceptive shortcut / causal confusion

This is the **OBJECT-BLINDNESS** — why it reaches the same spot regardless of the marker.

Plain language

The policy gets two kinds of input: **the joint state** (6 clean numbers describing where the arm is) and **the cameras** (two 224×224 images it must learn to interpret). The joint state is trivially easy to use and, in your demos, it is *correlated* with where the grasp ends up (because the arm always starts from the same home pose and follows similar paths). So the network takes the lazy route: it predicts the action **from the joint state** and barely uses the cameras. At deploy the arm always starts at the same home pose, so the joint state is always the same → the policy always produces the same “average” reach → it **ignores where the marker actually is**.

The mechanism, precisely

This is a specific case of **causal confusion** (de Haan, Jayaraman, Levine, NeurIPS 2019): BC learns $P(\text{action} \mid \text{observation})$, which is about *correlation*. It does not learn $P(\text{action} \mid \text{do}(\text{observation}))$, which is about *causation*. The cameras *cause* the correct reach (they say where the marker is); the joint state merely *correlates* with it in your particular dataset. BC cannot tell the difference, and the joint state is the cheaper signal, so it wins.

A profoundly counter-intuitive consequence proven in that paper: **adding more “helpful” state can make generalisation worse**, because it gives the network an even easier shortcut to exploit. Zhao et al. 2025 (“Do You Need Proprioceptive States in Visuomotor Policies?”) confirm it on real robots — *removing* proprioception entirely took pick-and-place spatial generalisation from **0% → 85%**.

How it produced YOUR symptom

- **Your own ablation is the proof:** the `config.joint_dino_prop.yaml` header states “*proprio influenced the action 2.2× more than vision.*” That is a direct measurement of the shortcut — the output moves 2.2× more when you perturb the joint state than when you perturb the image.
- Across rollouts the grasp pose lands in the same ~360–403 mm band regardless of the scene — a fixed *average* reach, exactly what an object-blind, state-driven policy produces.
- You correctly tried to fight it with **proprioception dropout 0.5** (zero the joint state half the time during training, forcing vision to contribute). That is the right instinct — but on its own it was not enough, because (a) the data still lets the shortcut form and (b) the CVAE collapse (file 3) reintroduced mean behaviour. See fixes in [file 4](#).

Canonical papers

- de Haan, Jayaraman, Levine, *Causal Confusion in Imitation Learning* (NeurIPS 2019).
arXiv:1905.11979
- Zhao et al., *Do You Need Proprioceptive States in Visuomotor Policies?* (2025).
arXiv:2509.18644

2.3 Mode averaging / regression to the conditional mean

This is **WHY** the grasp is a “mean” location — and it is the failure mode the CVAE was supposed to prevent.

Plain language

Your demonstrations are **multimodal**: the marker is in different places, and a human reaches slightly differently each time (different approach angle, different wrist twist, different exact grasp point). For a given camera view there is therefore **no single “right” action** — there are several. BC with a squared/absolute-error loss responds to “several right answers” by predicting **their average**. The average of “reach left” and “reach right” is “reach to the middle” — a spot where *no demonstration ever grasped*. So the arm reaches a phantom average location and closes on nothing.

The mechanism, precisely

It is a one-line statistical fact. The function that minimises mean-squared error is the **conditional mean**:

$$\pi^*(o) = \operatorname{argmin}_{\pi} E[(\pi(o) - a)^2] = E[a \mid o] \quad (\text{the average action for that observation})$$

So a plain regression BC policy is *mathematically guaranteed* to output the average of all valid actions for an observation. If those actions form two clusters (two modes), the average lands between them — often an invalid, do-nothing, or wrong-location action.

ACT’s intended cure — and why it failed here. ACT adds a **CVAE**: a latent variable z that encodes *which* demonstration “style/mode” is being used. During training the encoder infers z from the action sequence; at inference $z = 0$ (the prior mean) is used, which commits the decoder to **one** mode instead of averaging all of them. The ACT paper shows this matters enormously — removing the CVAE drops success from **35.3%** → **2%** on fine-manipulation tasks.

In your runs, the CVAE turned itself off. [File 3](#) shows the KL term collapsed to ~ 0 in every run — a phenomenon called **posterior collapse**. When the KL collapses, the latent carries no information; the decoder ignores z ; and ACT degenerates into exactly the plain regression that mode-averages. So the very mechanism meant to prevent §2.3 was silently disabled.

This is the most actionable single finding in the whole analysis.

How it produced YOUR symptom

- Object-blind “average” reach + mid-air grasp = textbook mode averaging.
- The training logs show **CVAE posterior collapse in all 5 ACT runs** (KL: 0.21 $\rightarrow$ 0.0001), i.e. ACT was running as plain mean-regression. (See file 3 §3.2.)

Canonical papers

- Chi et al., *Diffusion Policy* (RSS 2023) — the multimodality/averaging motivation. [arXiv:2303.04137](#)
 - Zhao et al., *ACT / Learning Fine-Grained Bimanual Manipulation* (RSS 2023) — the CVAE and the 35% $\rightarrow$ 2% ablation. [arXiv:2304.13705](#)
 - Florence et al., *Implicit Behavioral Cloning* (CoRL 2021). [arXiv:2109.00137](#)
 - Shafiullah et al., *Behavior Transformers (BeT)* (NeurIPS 2022). [arXiv:2206.11251](#)
-

2.4 Gripper-state ambiguity (premature / phantom grasp)

This is the MID-AIR GRASP and the “retract as if carrying something.”

Plain language

The policy decides when to close the gripper based on **where it is in the trajectory** (arm pose / timing), not on whether it is actually touching the marker — because nothing in its input tells it about contact. So it closes at the “usual” moment. Worse, its *only* sense of “do I have the object?” is the **binary gripper state** (closed = 1). A gripper closed on a marker and a gripper closed on air produce the **same** observation (state = “closed”). So once it closes on nothing, the policy *believes the grasp succeeded* and moves to the next phase — it starts the “transport” motion (you see the arm retract) while holding nothing.

The mechanism, precisely

Two coupled issues, formalised in “*Disambiguate Gripper State in Grasp-Based Tasks*” ([arXiv:2503.23835](#)):

1. **The close command is a learned temporal shortcut.** “Close at ~60–80% of the reach” is a reliable pattern in the data, so BC learns it as a function of arm progress, not contact.
2. **The binary gripper observation is ambiguous.** Three physical situations — *open-empty*, *closed-on-object*, *closed-on-air* — collapse into **two** observable values (open / closed). The policy literally cannot perceive the difference between a successful and a failed grasp, so it “transitions directly to the post-grasp stage, outputting a pull/transport action despite not having secured the object” (their words; your logs show exactly this).

How it produced YOUR symptom

- `deploy_ft.out` / `deploy_grip4.out`: [GRIPPER] CLOSED at **EEF 360–403 mm from start** (mid-air), then the arm **retracts** (387→313 mm, 364→285 mm) — the phantom “transport.”
- `deploy_dino*.out` (no gripper dropout): closes at **step 2** — the temporal shortcut at its most extreme (it learned “start closed”).
- `deploy_bowl.out`: after the mid-air half-close, the gripper command **oscillates 0.43–0.63 for 1329 steps** — it never even commits, because the gripper dimension is high-variance and unanchored to any real contact signal.

Canonical paper

- *Disambiguate Gripper State in Grasp-Based Tasks: Pseudo-Tactile as Feedback Enables Pure Simulation Learning* (2025). `arXiv:2503.23835`

2.5 Covariate shift / compounding error (why it never reaches the bowl)

This is the **LAST-MILE failure** — and why “offline looks fine, online is broken.”

Plain language

BC is trained only on the states a *successful expert* visits. At deploy, the policy makes small errors, which move it into slightly different states, where it makes bigger errors, and so on — errors **compound**. Once it does something the expert never did (close on air), it is in a state that **appears nowhere in the training data**. The “move to the bowl” behaviour in your data *always* begins from “object is in hand.” With no object in hand and an out-of-distribution state, the policy has no learned action for “what now?” — so it stalls or wanders, and the bowl phase never begins.

The mechanism, precisely

Ross & Bagnell (Dagger, 2011) proved the classic result: under BC the expected error grows like **$O(T^2)$** in the task horizon `T` (versus $O(T)$ if you could correct it interactively). A long multi-stage task (reach → grasp → transport → place) is exactly the worst case: an early-stage error puts you off-distribution and every later step amplifies it. Two extra effects make the grasp the apex of the problem:

- **Close-up OOD**: at grasp range the wrist camera sees a big, partly gripper-occluded view that is *under-represented* in the demos, so the policy is most uncertain exactly where it most needs to be precise.
- **The faulty offline metric**: validation loss is computed on expert states, so it cannot see the compounding online drift. That’s why your checkpoints have fine val loss yet fail online.

How it produced YOUR symptom

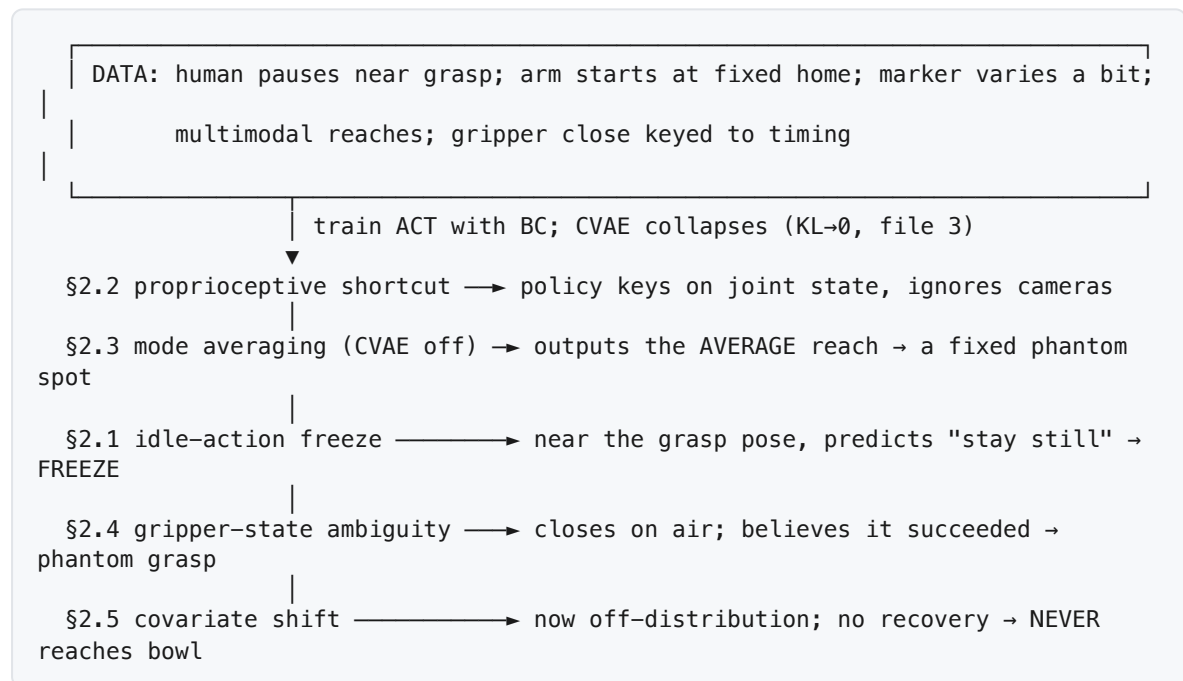
- Every rollout ends at the (failed) grasp — the transport/place phase never starts.
- Offline `diag_offline.npz` predictions look like sane small motions, and val loss is OK (0.144–0.174), yet the closed-loop rollouts consistently fail — the canonical offline-fine / online-broken signature of covariate shift.

Canonical papers

- Ross, Gordon, Bagnell, *A Reduction of Imitation Learning ... to No-Regret Online Learning* (DAgger, AISTATS 2011).
- Laskey et al., *DART: Noise Injection for Robust Imitation Learning* (CoRL 2017).
`arXiv:1703.09327`

2.6 How the five stack into the behaviour you saw

None of these acts alone — they **chain**:



The order matters for fixing it: the **proprioceptive shortcut + mode averaging** make it object-blind (reach the wrong place), the **idle-action problem** freezes it there, the **gripper ambiguity** fakes a grasp, and **covariate shift** prevents any recovery.

Continue to [03_training_log_findings.md](#) for the training-side evidence (especially the CVAE collapse), or jump to [04_root_cause_and_fixes.md](#) for what to do.

3. Training-Log Findings — what the training tells us (and the bug it hides)

File 1 and file 2 explained the *deploy* behaviour. This file looks at the **training logs** themselves (`experiment_data/training_logs/`) and finds the training-side causes — most importantly **one concrete bug** (the CVAE switched itself off) and several “this metric is lying to you” traps.

Logs analysed (5 ACT runs + 1 Octo run):

Log	Config
<code>train_dino_frozen.out</code>	DINOv2 fully frozen, no dropout
<code>train_dino_gripper_dropout.out</code>	+ gripper dropout 0.5
<code>train_dino_partial_unfreeze.out</code>	DINOv2 last block fine-tuned + gripper dropout
<code>train_dino_proprio_dropout.out</code>	+ proprio dropout 0.5 + gripper dropout 0.5 (this is the deployed model)
<code>train_act_abs_eef_resume.log</code>	ACT with absolute-EEF actions (resumed run)
<code>train_octo_2cam.log</code>	Octo base-1.5, 2-camera (different model — skimmed)

3.1 The headline: every ACT run trained “successfully” and still fails

There were **no crashes, no NaNs, no exploding losses, no obvious instability** in any ACT run. The losses go down smoothly and early-stopping triggers normally. **This is the trap:** the training looks healthy, so nothing screams “broken” — but the model it produces is object-blind. The problems are **systematic bias**, not numerical failure. You have to know what to look for.

3.2 THE BIG ONE: CVAE posterior collapse (the latent is dead)

This is the most important training finding and it directly explains the **mode averaging** (file 2 §2.3).

What you should see vs what happened

ACT’s loss has two parts: an **L1 reconstruction** term (predict the action) and a **KL** term (keep the CVAE latent `z` informative). A healthy CVAE keeps the KL at some **non-trivial, stable** value — that means `z` is actually carrying “which demo style” information.

In **every** ACT run, the KL term **collapses to ~zero**:

```
KL over epochs (representative, train_dino_frozen.out):
ep1    0.2107
ep2    0.0509
ep3    0.0171
ep5    0.0054
ep10   0.0012
ep15   0.0004
ep20   0.0002
ep30+  0.0001 ← essentially zero (the partial-unfreeze run prints 0.0000)
```

Halved every 1–2 epochs, effectively dead by epoch ~15–20. With `kl_weight = 10`, the KL’s contribution to the loss at that point is $10 \times 0.0001 = 0.001$ — negligible next to an L1 of ~0.08–0.15.

Why this is the whole ballgame

Posterior collapse = the CVAE latent `z` is being ignored. The encoder maps everything to the prior, the decoder learns to produce the action **without using `z`**. When that happens, ACT is no longer “a CVAE that commits to one mode” — it is just a **plain regression network**. And a plain regression network, by the math in file 2 §2.3, outputs the **average** action → the object-blind mean reach and the mid-air grasp.

In other words: the one architectural feature that was supposed to stop mode-averaging **turned itself off during training**, in every single run, and nobody noticed because the overall loss still went down. This is a fixable bug, not a fundamental limitation.

Likely cause: `kl_weight = 10` is high, which (combined with a powerful decoder) is a classic recipe for posterior collapse — the optimiser finds it cheaper to drive KL to zero than to use the latent. Fixes (KL annealing, lower weight, free-bits) are in [file 4 §4.1](#).

3.3 🟡 The validation loss is lying to you (anti-correlation with deploy success)

This one is operationally dangerous because it means **your model-selection criterion is wrong**.

Run	Best val_l1	Deployed?	Deploy result
dino_frozen	0.1444 (lowest)	not the deployed one	—
gripper_dropout	0.1471	—	—
partial_unfreeze	0.1485	—	—
abs_eef_resume	0.1586	—	—
proprio_dropout	0.1738 (highest)	yes (epoch 19)	the failure you reported

The model you actually deployed had the **worst** validation loss of the bunch, and it fails the same way the “best val loss” model would. **Validation loss does not predict deploy success here — if anything it is inverted.**

Why: a *lower* val loss can be achieved by fitting the training distribution’s **mean pose** more tightly. Since the mean pose *is* the (object-blind) failure behaviour, “better val loss” can literally mean “better at being object-blind.” No threshold on val_l1 can distinguish “uses vision and generalises” from “predicts the average well.”

Consequence: stop picking checkpoints by val_l1. Pick them with a **causal-intervention probe** (does the action follow the image or the state?) or a **scored real/sim rollout**. See [file 4 §4.5](#).

3.4 🟡 The val-loss plateau is the “object-blind ceiling”

In every run, **train_l1 keeps dropping** (to ~0.05) while **val_l1 plateaus** at ~0.14–0.17 and basically stops improving:

```
train_dino_frozen:  val_l1  0.220 → 0.149 (ep8) → 0.147 (ep15) → 0.1444 (ep28) →  
0.149 (ep40)  
                    train_l1 0.269 → ..... → 0.059 (ep40)
```

- The **persistent gap** (val – train $\approx$ 0.077–0.105 across runs) on a 72–84M-parameter model trained on ~86 episodes is **memorisation**: the network fits the training trajectories but does not learn the visually-conditioned task.
- The **plateau height** (~0.14–0.17) barely changes across *every* config — frozen, fine-tuned, proprio-dropout, gripper-dropout. That stable ceiling is the residual error of the **object-blind mean predictor**. The model cannot get under it because getting under it would require *using vision to localise the marker*, which it has learned not to do.

A subtle but important point about “low loss looks good”

train_l1 $\approx$ 0.05 *looks* excellent. But for **absolute** 7-D joint/gripper actions, “predict $\approx$ the current pose / the trajectory mean” already yields a low L1 — so a low number here is consistent with the **idle-action / predict-current-pose shortcut** (file 2 §2.1), not with task competence. Low L1 is necessary but nowhere near sufficient.

3.5 🟡 Did the dropout regularisers actually do anything?

You added two regularisers to fight the shortcut. The logs show whether they “bit”:

Regulariser	Epoch-1 train_l1 vs frozen (0.269)	Verdict
Proprio dropout 0.5	0.309 (~13% higher)	Worked as intended — masking the joint state genuinely made the task harder, which is exactly what you want (it stops the policy from leaning on state for those samples). It also produced the highest, most <i>honest</i> val_l1 (0.1738) and the earliest stop (ep31), i.e. it couldn't overfit as deeply.
Gripper dropout 0.5	0.274 (~2% higher)	Did almost nothing — masking the gripper dimension barely changed the loss, because the model was barely using the gripper signal in the first place. Dropping a signal the model ignores has no regularising effect.

So proprio dropout is pulling in the right direction (keep it), but it is **not sufficient alone** because (a) the data still permits the shortcut and (b) the CVAE collapse reintroduced mean behaviour. Gripper dropout is mostly a no-op for the loss — the gripper problem needs a **contact/force signal**, not dimension-masking (file 4 §4.4).

3.6 🟡 Other weird / notable things in the logs

Finding	Detail	Why it matters
Corrupted frames	14 frames permanently replaced with zeros (same indices every epoch) across 11 train + 3 val episodes — e.g. ep82/idx1010 , ep44/idx945 , ...	~14% of episodes have a zeroed visual frame at a fixed timestep — minor but real data poisoning; worth fixing the converter / re-extracting those frames.
“Frozen” vs “partial_unfreeze” not capacity-matched	frozen = 72.7M trainable; partial_unfreeze = 74.5M (the extra 1.8M = the unfrozen DINOv2 last block at 0.1× LR)	The comparison between those two runs is muddled by a capacity difference — keep that in mind when reading their val numbers.
abs_eef run	resumed at epoch 5; smaller dataset (70 train / 17 val vs 86/21); best val_l1 0.1586 @ ep36; KL already collapsed by ep5	Absolute-EEF didn't help and the log is missing epochs 1–5; its higher val is partly the smaller dataset.
Octo 2-cam run	trained 15k steps cleanly; only issue is a TensorFlow not built for compute capability 12.0 → JIT-from-PTX warning (slow ~9 s first step) and benign CropAndResize PredictCost messages	No training instability; the GPU is newer than the prebuilt TF supports (Ada/Blackwell-class), so TF JIT-compile kernels at startup. Cosmetic/perf only.
No step-loss for Octo	WandB in offline mode → per-step losses are in WandB, not the log	Can't assess Octo's loss curve from the log alone.

3.7 Cross-run comparison table

Config	Best val_l1	Best epoch	Final train_l1	KL @ best ep	Early stop @	val – train gap
dino_frozen	0.1444	28	0.059 (ep40)	~0.0001	40	~0.077
gripper_dro pout	0.1471	40	0.054 (ep52)	~0.0001	52	~0.093
partial_unfreeze	0.1485	53	0.049 (ep65)	0.0000	65	~0.100
abs_eef_resume	0.1586	36	0.053 (ep48)	~0.0001	48	~0.105
proprio_dro pout (deployed)	0.1738	19	0.067 (ep31)	~0.0001	31	~0.104

Read this table as: every run collapsed the KL, every run has a big memorisation gap, the val numbers are all clustered around the object-blind ceiling, and the deployed run is the one with the highest (worst) val loss — confirming val loss is not the right selector.

3.8 The four training-side verdicts (direct answers)

1. **Is there CVAE posterior collapse? Yes — completely, in every run.** KL 0.21 $\rightarrow$ ~ 0.0001 . ACT was running as plain regression. ($\rightarrow$ mode averaging, file 2 §2.3.) **This is the top bug.**
2. **Is the L1 suspiciously low / idle-action-shaped? Yes.** train_l1 ~ 0.05 with a val plateau ~ 0.15 is consistent with “predict $\approx$ current pose,” not task competence.
3. **Did the dropout regularisers change the loss as expected? Proprio dropout: yes** (+13% train loss, the right direction). **Gripper dropout: no** (+2%, basically a no-op).
4. **Does best val_l1 correlate with deploy success? No — it is inverted.** The deployed model had the worst val loss. Don't select on val loss.

Next: [04_root_cause_and_fixes.md](#) turns all of this into a prioritized action plan.

4. Root Cause and Fixes — what to actually do

This is the “do something” file. It states the **root-cause chain** in one place, then gives a **prioritized fix list**: what to change, why, how to do it concretely, and how to **verify** it worked. Fixes are ordered by *impact per unit effort* given the evidence — do them top to bottom.

If you read only one thing: **fix the CVAE collapse (§4.1) and fix the data (§4.2) first**. Everything else is secondary until those two are done.

4.0 The root-cause chain, in one paragraph

The demonstrations let the policy predict the action from the **joint state** instead of the **cameras** (proprioceptive shortcut), and ACT’s defense against averaging — the **CVAE latent** — **collapsed during training** ($KL \rightarrow 0$), so the policy became a plain regressor that outputs the **average** demonstrated motion. That average is **object-blind** (reaches a fixed phantom spot), **freezes** near the grasp because pause frames taught it “stay still there” (idle-action), and **closes the gripper on air** because the close is timed, not contact-triggered, and the binary gripper state can’t tell a real grasp from a fake one. Once it fakes the grasp it is **out-of-distribution** and never recovers to the place phase. Fix the *data* and the *CVAE* and most of this disappears; fix the *gripper signal* and the *idle frames* to finish the job.

ROOT CAUSES (fix these)		SYMPTOMS (these then go away)
data lets state predict the grasp spot	→	object-blind reach to a fixed
CVAE posterior collapse ($KL \rightarrow 0$)	→	mean / averaged motion
absolute-joint actions + idle frames	→	freeze near the grasp
gripper close is timed, state is binary	→	mid-air grasp + phantom transport
all of the above → off-distribution	→	never reaches the bowl

4.1 FIX #1 — Stop the CVAE posterior collapse (training bug; highest leverage)

Why first: it is a concrete, certain bug ($KL \rightarrow 0$ in every run, [file 3 §3.2]); fixing it re-enables the one mechanism that prevents mode-averaging; and it is a few-line change.

What to do (any/all of): - **Lower `kl_weight`** from `10` to roughly `0.1–1.0`. A weight of `10` over-penalises the latent and is a classic collapse trigger. - **KL annealing / warm-up:** start `kl_weight` near `0` and ramp it up over the first few thousand steps, so the decoder learns to use `z` before it is pressured to shrink it. - **Free bits:** floor the KL per latent dimension (e.g. `max(KL, 0.1)`) so the optimiser cannot drive it to exactly zero.

How to verify it worked: plot the KL over training — it should **stabilise at a non-trivial value** (not crater to ~ 0.0001). Then do a quick check that the latent matters: with two different `z` samples the predicted action chunk should differ noticeably for the same observation. If KL

stays healthy and `z` changes the output, the CVAE is alive again.

Reference: ACT relies on the CVAE (removing it drops success 35%→2%, Zhao et al. 2023); posterior collapse is a well-known VAE failure with high KL weight (Bowman et al. 2016, “Generating Sentences from a Continuous Space” — the origin of KL annealing).

4.2 FIX #2 — Fix the DATA so vision *has* to be used (highest leverage, more effort)

Why: the proprioceptive shortcut (file 2 §2.2) is a property of the **data**, not the model. No architecture change fully fixes it if the data still lets the joint state predict the grasp.

What to do: - **Fixed home + widely varied marker.** Start every demo from the *same* home pose, and place the marker across the **full reachable workspace** (a grid is ideal — e.g. 4×4 or 5×5 cells, several demos per cell). Now the start state carries **zero** information about where to grasp, so the only way to lower the loss is to **look at the camera**. This is the single most important data change. - **More demonstrations.** ~86 episodes is small; target **100–150+** with the varied-marker protocol. Below ~100 the cheap state shortcut almost always wins. - **Vary the bowl/target position too**, so the place phase also has to be vision-conditioned. - **Filter idle/pause frames** (this also helps the freeze, §4.3).

How to verify: after retraining, run the **causal-intervention probe** (you already have one — the `diag_ablation / predict_with_diag` instrumentation): blank the cameras vs. perturb the state and compare how much the action changes. Today proprio is **2.2x > vision**; the target is **image-dominant** (the action should change *more* when you blank the cameras than when you perturb the state). That number is your success metric for this fix.

References: de Haan et al. 2019 (causal confusion); Zhao et al. 2025 (state-free, 0%→85% from exactly this kind of change).

4.3 FIX #3 — Kill the freeze: remove idle frames + use relative actions

Why: the freeze is the idle-action problem (file 2 §2.1), amplified by the absolute-joint action space.

What to do: - **Filter idle/pause frames** from the dataset before training: drop frames where the per-step joint motion is below a small threshold ($\|\Delta q\| < \epsilon$) or where the action repeats the previous one for N frames. The Diffusion Policy paper shows this is *the* fix for “policy gets stuck.” Be careful to keep a few low-motion frames around genuine grasp moments so the

policy still learns to slow down — just don't let pauses dominate. - **Switch to relative actions** (delta joint or, better, **delta end-effector**). In delta space “do nothing” is the **zero vector**, which is unambiguous and easy to learn *not* to over-predict; in absolute space “do nothing” = “predict my current pose,” which is the cheap shortcut that feeds the freeze. (Your repo already has the `--action-space delta_eef` path; this is one of its benefits. Note delta-EEF also needs the IK deploy path — see `docs/action_spaces.md`.)

How to verify: in a rollout, the per-step commanded motion near the grasp should **not** collapse to ~ 0 ; and `mean |chunk0 - state|` (from `diag_offline`) should no longer be $\sim 0.84^\circ$ of “stay put” when the task wants motion.

References: Chi et al. 2023 (remove idle actions); Wen et al. 2020 (copycat — relative/short history breaks the shortcut).

4.4 FIX #4 — Make the grasp contact-aware (stop the mid-air close + phantom transport)

Why: the gripper closes on timing and the binary state can't tell a real grasp from a fake one (file 2 §2.4).

What to do: - **Gate the transport phase on an actual grasp signal.** You have `gripper_norm` in the 7-D state already; even better, read the **gripper width / force** at deploy. After a close command, check: did the gripper *fail to fully close* (object present, width stays open) or *close all the way* (likely empty)? Only proceed to “move to bowl” if the grasp looks real; otherwise **re-attempt** the approach. This is a deploy-side closed-loop check, independent of the policy. - **Add a contact/force or pseudo-tactile observation** so the policy itself can perceive “holding vs empty” (the three states open-empty / closed-on-object / closed-on-air become distinguishable). The cited paper does this with the force-gripper's own joint angle — no extra hardware. - **Collect more grasp-phase data** (close-up, gripper-near-object views) so the contact moment is in-distribution.

How to verify: in rollouts the gripper should only commit to “closed → transport” when a grasp is actually detected; mid-air closes should trigger a retry, not a phantom carry.

Reference: *Disambiguate Gripper State in Grasp-Based Tasks* (arXiv:2503.23835).

4.5 FIX #5 — Stop selecting models by validation loss

Why: `val_l1` is anti-correlated with deploy success here (file 3 §3.3) — the deployed model had the *worst* val loss.

What to do: - **Select checkpoints by the causal-intervention probe** (image-vs-state dependence) and/or a **scored rollout** (sim or real), not by val_l1. - Track the **vision-dependence ratio** as a first-class metric during training (you already compute it). A model isn't "better" because its val loss is lower; it's better because it *uses the cameras*.

How to verify: the metric you optimise should move with real rollout success. If lowering val_l1 doesn't improve rollouts, you're tuning the wrong number.

4.6 FIX #6 (structural, if the above isn't enough) — change the policy class

Why: ACT-with-collapsed-CVAE mode-averages; if you can't keep the CVAE healthy, use a policy that handles multimodality natively.

Options: - **Diffusion Policy** — denoises to a *single* mode instead of averaging; the Diffusion Policy paper shows it doesn't even need idle-frame removal to avoid getting stuck. - **Octo (finetuned)** — you already have this wired (`policies/octo/`). It is vision+language, no proprioceptive state input by default — so it **structurally avoids the proprioceptive shortcut**, and it brings an 800k-trajectory visual prior, which is exactly what a ~86-episode dataset lacks. Strong candidate to compare against the fixed ACT.

References: Chi et al. 2023 (Diffusion Policy); Octo docs in this repo (`docs/octo*.md`).

4.7 The priority checklist (tear-off version)

```
[ ] 1. CVAE: lower kl_weight to ~0.1–1.0 + KL anneal/free-bits; verify KL stays > 0 and z matters
[ ] 2. DATA: re-record fixed-home + varied-marker (grid), 100–150+ demos, vary bowl too
[ ] 3. FREEZE: filter idle/pause frames; move to relative (delta-EEF) actions
[ ] 4. GRASP: deploy-side grasp-success gate (gripper width/force) + add contact obs; more grasp data
[ ] 5. SELECTION: pick checkpoints by the vision-vs-state probe / scored rollout, NOT val_l1
[ ] 6. (if needed) try Diffusion Policy or finetuned Octo as a multimodal/state-free alternative
[ ] re-run the causal-intervention probe after each change – target: image-dominant (not 2.2x state)
```

Expected payoff per fix (rough, evidence-based)

Fix	Attacks	Expected effect
#1 CVAE	mode averaging	restores ACT's anti-averaging mechanism; less "mean" behaviour
#2 Data	proprioceptive shortcut	the big one — makes the reach actually track the marker
#3 Idle+relative	freeze	arm keeps moving through the grasp instead of stalling
#4 Grasp gate	mid-air grasp + no-bowl	real grasps; retries instead of phantom transport; reaches bowl
#5 Selection	choosing a bad model	you stop shipping the object-blind checkpoint
#6 Policy class	residual averaging / small data	structural robustness if ACT still struggles

Validation north-star: the single number that summarises whether you've fixed the core problem is the **vision-vs-state dependence ratio** from your own probe. It is **2.2× toward state** today. When it flips to **image-dominant**, the object-blindness — and most of the cascade — is gone.

See [05_glossary_and_references.md](#) for definitions and all paper links.

5. Glossary and References

Plain-language definitions of every term used in this folder, then all the papers with links.

5.1 Glossary

ACT (Action Chunking Transformer) — the policy here. A transformer that, from the current observation, predicts a *chunk* of many future actions at once (here 100 steps), wrapped in a CVAE to handle demonstration variability. Paper: Zhao et al. 2023.

Action chunk — a block of consecutive future actions predicted in one shot (vs. one action per step). Reduces the effective decision horizon, which helps with compounding error.

Behavioral cloning (BC) — training a policy by supervised learning on (observation, action) pairs from demonstrations. “Copy what the human did.” No goal, no feedback, no causal reasoning.

Causal confusion — BC learns *correlations* between observations and actions, not *causes*. It can latch onto an input that merely correlates with the right action (joint state, previous action) and ignore the input that actually determines it (the camera). Paper: de Haan et al. 2019.

Compounding error — small per-step mistakes push the robot into states the expert never visited, where it makes bigger mistakes, which compound. Under BC the total error grows like $O(T^2)$ in the task length T . Paper: Ross et al. 2011 (DAgger).

Conditional mean — the average action for a given observation, $E[a|o]$. The mathematical answer that mean-squared-error regression converges to — which is why plain BC mode-averages.

Copycat / inertia problem — the policy learns the shortcut “output $\approx$ my previous action,” because consecutive expert actions are nearly identical. Near a slow-down it self-perpetuates “keep (almost) stopping,” producing a freeze. Paper: Wen et al. 2020.

Covariate shift — the train-time state distribution (expert states) differs from the deploy-time state distribution (states the policy’s own errors lead it into). The core reason BC fails online while looking fine offline.

CVAE (Conditional Variational Autoencoder) — ACT’s mechanism for multimodality. A latent variable z encodes “which demonstration style/mode” so the decoder can commit to *one* mode instead of averaging. Controlled by a **KL** term during training.

DINOv2 — a large self-supervised Vision Transformer (from Meta). Used here as a **frozen** image feature extractor in place of ACT’s trainable ResNet18. Strong general visual features without training them on your small dataset.

EEF / TCP — End-Effector / Tool Center Point: the position+orientation of the gripper in 3-D space. “EEF displacement” = how far the tool tip moved.

Idle-action problem — demonstrations contain “stay still” frames (operator pauses, especially near the grasp). BC over-learns these as a confident “do nothing” rule, so the robot freezes there. Paper: Chi et al. 2023 (Diffusion Policy) names it.

KL term / KL divergence — in the CVAE loss, the term that keeps the latent z informative and close to its prior. If it collapses to ~ 0 , the latent is unused → **posterior collapse**.

Mode / multimodal — a “mode” is one valid way to do the task (reach left vs. right; marker here vs. there). Demonstrations are “multimodal” when several modes appear. BC averages them.

Mode averaging — predicting the average of multiple valid actions, which often lands on an invalid in-between action (reach to the middle of two grasp points → grasp nothing).

Posterior collapse — a VAE/CVAE failure where the latent is ignored ($KL \rightarrow 0$). Here it turned ACT into a plain regressor → mode averaging. Classic with too-high KL weight. Origin of the fix (KL annealing): Bowman et al. 2016.

Proprioception / proprioceptive state — the robot’s sense of its own body: joint angles, gripper opening, end-effector pose. The 7-D state input here (6 joints + gripper).

Proprioceptive shortcut — a specific causal confusion: the policy predicts the action from the (clean, easy) joint state and ignores the (hard) cameras, so it reaches the same average spot regardless of where the object is. Paper: Zhao et al. 2025.

Proprioception dropout — randomly zeroing the joint-state input during training so the policy can’t rely on it and is forced to use vision. You used rate 0.5. (`common/proprio.py`)

Temporal ensembling — ACT’s inference smoothing: query the policy every step and average the overlapping predicted chunks with exponentially decaying weights. Here it *helps* (turning it off made motion worse); it is **not** the cause of the freeze.

Validation loss (val_l1) — the prediction error on held-out expert frames. Here it is **anti-correlated** with deploy success (the worst-val model was the deployed one), because a lower val loss can just mean “better at predicting the object-blind mean.”

5.2 References (all papers cited across this folder)

The core failure modes

- **Causal confusion / proprioceptive shortcut**
 - de Haan, Jayaraman, Levine — *Causal Confusion in Imitation Learning* — NeurIPS 2019 — [arXiv:1905.11979](https://arxiv.org/abs/1905.11979)
 - Zhao et al. — *Do You Need Proprioceptive States in Visuomotor Policies?* — 2025 — [arXiv:2509.18644](https://arxiv.org/abs/2509.18644) · [project](#)
- **Idle-action problem & mode averaging**
 - Chi et al. — *Diffusion Policy: Visuomotor Policy Learning via Action Diffusion* — RSS 2023 — [arXiv:2303.04137](https://arxiv.org/abs/2303.04137)

- **Copycat / inertia**
 - Wen et al. — *Fighting Copycat Agents in Behavioral Cloning from Observation Histories* — NeurIPS 2020 — [arXiv:2010.14876](#)
 - Chuang et al. — *Resolving Copycat Problems in Visual Imitation Learning via Residual Action Prediction* — ECCV 2022 — [arXiv:2207.09705](#)
- **Covariate shift / compounding error**
 - Ross, Gordon, Bagnell — *A Reduction of Imitation Learning ... to No-Regret Online Learning (DAGger)* — AISTATS 2011 — [PDF](#)
 - Laskey et al. — *DART: Noise Injection for Robust Imitation Learning* — CoRL 2017 — [arXiv:1703.09327](#)
- **Gripper-state ambiguity / premature grasp**
 - *Disambiguate Gripper State in Grasp-Based Tasks: Pseudo-Tactile as Feedback Enables Pure Simulation Learning* — 2025 — [arXiv:2503.23835](#)

The model and its mechanisms

- **ACT (the policy)**
 - Zhao, Kumar, Levine, Finn — *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware* — RSS 2023 — [arXiv:2304.13705](#)
- **CVAE posterior collapse / KL annealing**
 - Bowman et al. — *Generating Sentences from a Continuous Space* — CoNLL 2016 — [arXiv:1511.06349](#)
- **Multimodality alternatives**
 - Florence et al. — *Implicit Behavioral Cloning* — CoRL 2021 — [arXiv:2109.00137](#)
 - Shafiullah et al. — *Behavior Transformers: Cloning k Modes with One Stone* — NeurIPS 2022 — [arXiv:2206.11251](#)

Related repo docs (this repository)

- [docs/act.md](#) — ACT architecture from scratch (CVAE, $z=0$, temporal ensembling).
- [docs/il_failure_modes.md](#) — the broader catalogue of BC failure modes.
- [docs/proprioception_modes.md](#) — the full/dropout/none state switch.
- [docs/action_spaces.md](#) — joint vs delta-EEF actions + the IK deploy path.
- [docs/octo*.md](#) — the state-free, pretrained alternative policy.

5.3 A note on rigor

Every claim in this folder is tied either to a **measured number from your logs** (`experiment_data/...` , cited inline) or to a **named, real paper** (listed above). Where a fix's payoff is an estimate rather than a measurement, it is labelled "rough/expected." The single most important *measured* fact is your own ablation — **proprio influenced the action 2.2x more than vision** — and the single most important *training* fact is the **CVAE posterior collapse** (KL $\rightarrow \sim 0$ in every run). Those two, together, explain the object-blind average behaviour; the idle-action freeze, the mid-air grasp, and the never-reaching-the-bowl follow from there.

6. The CVAE KL Term — a from-scratch deep dive

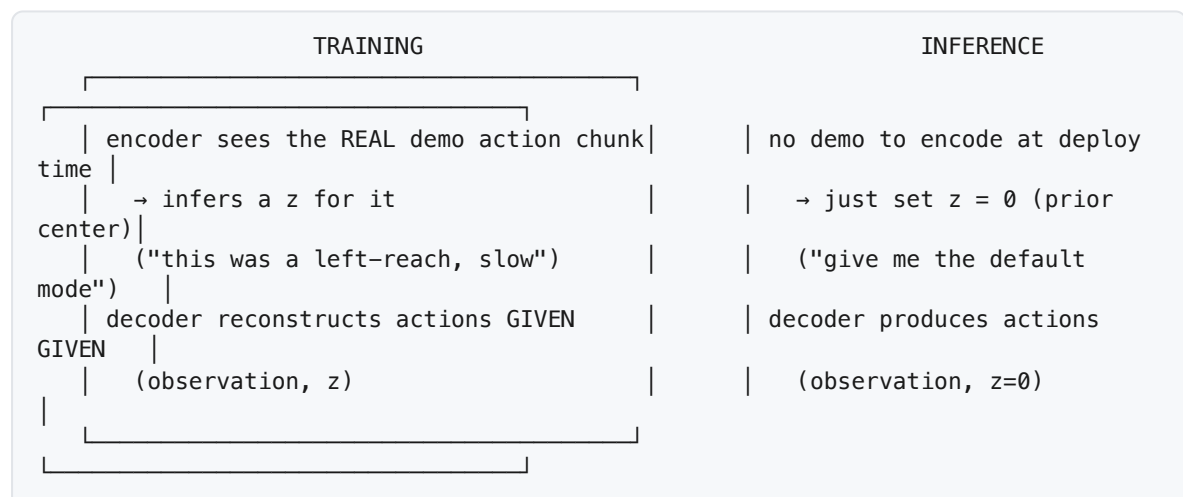
This file zooms all the way in on the single most important *training* finding in this folder: the **KL term of ACT's CVAE collapsed to ~0** ("posterior collapse"), which silently turned ACT into a plain regressor that mode-averages. To understand that, you need to understand what the KL term *is*, why it must be there, and why it died. This file builds it up from zero.

Cross-refs: the consequence (mode averaging) is [file 2 §2.3](#); the measured collapse is [file 3 §3.2](#); the fixes are [file 4 §4.1](#).

6.1 Why ACT has a latent `z` at all

ACT's core problem: **for one observation there are many valid action sequences** — a human grasps slightly differently every time (different approach angle, speed, exact grasp point). A plain network forced to output *one* answer for *many* valid ones outputs their **average** (mode averaging — and the average of "reach left" and "reach right" is "reach to the middle," which grasps nothing).

The CVAE's fix is to add a **latent variable `z`** (32-dim in your config: `latent_dim: 32`) that encodes "*which style/mode is this demonstration.*"



Because the decoder is *told which mode* via `z`, it never has to average. **But this only works if `z` actually carries the mode information.** The KL term is what forces `z` to do that — and what makes `z = 0` a meaningful choice at inference.

6.2 The CVAE loss = reconstruction + KL

ACT is trained to maximise the **ELBO** (evidence lower bound), which as a loss is:

$$L = \underbrace{\text{reconstruction_loss}}_{\text{the L1 term}} + \underbrace{\beta \cdot \text{KL_term}}_{\beta = \text{kl_weight} = 10 \text{ in your config}}$$

- **Reconstruction loss** — “did the decoder reproduce the demonstrated actions?” (your L1).
- **KL term** — a regulariser on the latent. **This is the part you asked about.**

The β (= `kl_weight`) is the knob that sets the balance between the two. Your runs used `kl_weight = 10`, which — as we’ll see — is a big part of why it collapsed.

6.3 What the KL term actually is (the math, explained)

The encoder does **not** output a single z . It outputs a **distribution** over z — a Gaussian with a mean μ and standard deviation σ predicted from the demo:

$$q(z \mid \text{demo}) = \text{Normal}(\mu, \sigma^2) \quad \leftarrow \text{the "posterior": the } z \text{ this specific demo implies}$$

The **prior** is a fixed standard Gaussian:

$$p(z) = \text{Normal}(0, 1) \quad \leftarrow \text{the "prior": } z \text{ before seeing any demo}$$

The **KL term** is the **KL divergence** between those two — a single number measuring *how far the encoder’s predicted distribution is from the standard Gaussian*:

$$\text{KL_term} = \text{KL}(q(z \mid \text{demo}) \parallel p(z))$$

For two Gaussians this has a clean closed form (per latent dimension, summed over all 32):

$$\text{KL} = \frac{1}{2} \cdot \sum_i (\mu_i^2 + \sigma_i^2 - 1 - \log \sigma_i^2)$$

Read the formula intuitively:

Situation	KL value	Meaning
$\mu = 0$, $\sigma = 1$ (posterior = prior)	0	the encoder output is identical to the prior → z says nothing about the demo
μ pushed away from 0	grows	the encoder uses the latent <i>mean</i> to encode demo-specific info
σ shrunk below 1	grows	the encoder is <i>confident/precise</i> about this demo’s z

So the KL term is a direct measure of **how much information the encoder is packing into z** :

- **KL large** → posterior far from prior → different demos get **different** z → z carries real mode information. **Good.**
- **KL ≈ 0** → posterior $\approx$ prior for *every* demo → the encoder outputs the same standard-Gaussian noise no matter what → z carries **nothing**. **This is the collapse.**

The reparameterisation trick (one-line aside)

You can't backprop through "sample z from $\text{Normal}(\mu, \sigma)$ ". So ACT samples noise $\epsilon \sim \text{Normal}(0, 1)$ and computes $z = \mu + \sigma \cdot \epsilon$. The randomness lives in ϵ (no gradient needed); gradients flow through μ and σ . That's just *how* the latent is made trainable — not the bug, but it's why μ , σ are learnable quantities the KL can act on.

6.4 Why the KL term has to be there (the tug-of-war)

The two loss terms pull z in **opposite directions**:

Term	Wants z to...	Why
Reconstruction (L1)	carry lots of info	the more z tells the decoder, the easier to reconstruct the exact demo → lower L1
KL	carry no info ($\mu \rightarrow 0, \sigma \rightarrow 1$)	the closer the posterior is to the prior, the lower the KL

A **healthy** CVAE settles in the middle: z carries *just enough* information to capture the demo's **mode/style** (not every pixel-level detail), with a smooth, prior-shaped latent space. $\beta = \text{kl_weight}$ sets where that balance lands — higher β pushes harder toward "z carries nothing."

There is a second, subtler reason the KL must be there: **it is what makes $z = 0$ valid at inference.** Training pulls every demo's posterior toward the $\text{Normal}(0, 1)$ prior, so the prior's center $z = 0$ becomes a meaningful "average/default mode" the decoder knows how to decode. Remove the KL and the latent space becomes arbitrary — $z = 0$ would point at garbage.

6.5 Posterior collapse — what went wrong in your runs

Posterior collapse = the KL side of the tug-of-war wins *completely*. The encoder learns to output $\mu \approx 0, \sigma \approx 1$ for **every** demo — i.e. exactly the prior — so $\text{KL} \rightarrow 0$. Your logs show this precisely, in every ACT run:

```
KL over epochs (train_dino_frozen.out, representative of ALL runs):
ep1    0.2107
ep2    0.0509
ep3    0.0171
ep5    0.0054
ep10   0.0012
ep15   0.0004
ep20   0.0002
ep30+  0.0001    ← dead (the partial-unfreeze run prints a literal 0.0000)
```

Halved every 1–2 epochs, effectively dead by ~epoch 10–15. With `kl_weight = 10`, the KL’s contribution to the total loss at that point is $10 \times 0.0001 = 0.001$ — negligible next to an L1 of ~0.08–0.15. The latent is gone.

What that does to the model

```
z carries no info → decoder learns z is useless noise → decoder ignores z
entirely

ALONE
      |
      | ACT now predicts actions from the observation
      |
      | = a plain regression network → outputs E[action |
obs]
      |
      | = the AVERAGE action → mode averaging (file 2
$2.3)
      |
      | object-blind "mean" reach + mid-air grasp (your deploy
logs)
```

In one sentence: the one architectural feature meant to *prevent* averaging (**the CVAE latent**) **erased itself during training**, in every run, and nobody noticed because the overall loss still went down smoothly.

6.6 Why it collapsed (the cause, concretely)

Two ingredients, both present in your setup:

1. **High `kl_weight = 10`**. A large β over-weights “push the posterior to the prior,” so the optimiser finds it *cheaper to zero the KL* than to learn to use `z`. This is the classic, well-documented trigger for posterior collapse (Bowman et al. 2016, who invented the standard fix while training text VAEs).
2. **A powerful decoder**. ACT’s transformer decoder is strong enough to reconstruct the action chunk *reasonably well without* `z`. When the decoder can “do fine” ignoring the latent, the reconstruction term doesn’t fight to keep `z` alive, so the KL pressure wins unopposed.

There’s also a **chicken-and-egg** dynamic at the start of training: before the decoder has learned to *use* `z`, the latent genuinely looks like useless noise — so the KL term happily shrinks it toward zero in the first few epochs, and then the decoder never gets a reason to

start using it. Once collapsed, it stays collapsed. (That’s why yours died by ~epoch 10, never to recover, in every run.)

6.7 How to keep the KL healthy (the fixes)

All three break the “KL wins instantly” dynamic. Use one or combine them.

Fix	What it does	Why it works
Lower <code>kl_weight</code> (10 → ~0.1–1.0)	reduce β so the latent isn’t crushed	less pressure toward the prior → <code>z</code> survives and stays informative
KL annealing / warm-up	start $\beta \approx 0$, ramp it up over the first few thousand steps	lets the decoder <i>first learn to use</i> <code>z</code> while it’s free, <i>then</i> gently regularise — so it can’t collapse before it’s useful (Bowman et al.’s original fix for exactly this)
Free bits	floor the KL per dimension, e.g. <code>KL_used = max(KL, λ)</code> with $\lambda \approx 0.1$	the optimiser gets no reward for pushing KL below the floor, so it physically cannot drive it to zero

How to confirm it worked

1. **Plot KL over training** — it should **stabilise at a non-trivial value** (roughly 0.5–5), not crater to ~0.0001.
2. **Sanity-check the latent is alive** — feed the *same* observation with two different `z` samples; the predicted action chunk should **visibly differ**. If `z` changes the output *and* KL stays > 0, the CVAE is back and ACT can commit to a mode instead of averaging.

6.8 One-paragraph summary

The KL term measures how much real information ACT’s latent `z` carries — formally the KL divergence between the encoder’s per-demo Gaussian posterior $q(z|\text{demo}) = \text{Normal}(\mu, \sigma^2)$ and the standard-normal prior $p(z) = \text{Normal}(0, 1)$, which equals $\frac{1}{2}\Sigma(\mu^2 + \sigma^2 - 1 - \log \sigma^2)$ and is zero only when the posterior equals the prior ($\mu=0, \sigma=1$). It must stay non-trivial for `z` to encode *which mode* a demo is — and for `z=0` to be a meaningful default at inference — so the decoder can commit to one mode instead of averaging. In your runs, `kl_weight=10` plus a strong transformer decoder drove the KL to ~0 within ~10 epochs in every run (**posterior collapse**): the latent went dead, the decoder ignored it, and ACT degenerated into a plain mean-predicting regressor — which is the root training cause of the object-blind, mode-averaged behaviour you saw on the robot. Fix it with a lower KL weight, KL annealing, and/or free bits, and verify the KL stabilises above zero and the latent actually changes the output.

References for this file

- Kingma & Welling — *Auto-Encoding Variational Bayes* (the VAE / ELBO / reparameterisation) — 2013 — [arXiv:1312.6114](#)
- Sohn, Lee, Yan — *Learning Structured Output Representation using Deep Conditional Generative Models* (the CVAE) — NeurIPS 2015
- Bowman et al. — *Generating Sentences from a Continuous Space* (posterior collapse + KL annealing) — CoNLL 2016 — [arXiv:1511.06349](#)
- Zhao et al. — *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware* (ACT; the CVAE and the 35%→2% ablation) — RSS 2023 — [arXiv:2304.13705](#)
- See also [docs/act.md](#) §6 (the CVAE loss) in this repo.

7. Grasp-Success Gate — requirements (deferred, needs a sensor signal)

One of the deploy-side failures (file 2 §2.4) is the **phantom grasp**: the policy closes the gripper in mid-air, then — because its only sense of “am I holding something?” is the *binary* gripper command — believes it succeeded and transitions to the transport phase, retracting with nothing in hand.

The fix is a **grasp-success gate**: after a close command, *verify a real grasp* before allowing the arm to proceed; otherwise re-attempt the approach. This is **not implemented** in this branch — not because of code difficulty, but because **the FR5 gripper exposes no feedback to gate on**.

Why it can't be coded yet

The robot controller (`../so101-fr5-teleop/fr5.py`) is **command-only** for the gripper:

Available	Method
read joint angles	<code>get_joint_positions()</code>
read TCP/EEF pose	<code>get_eef_pose()</code>
read joint velocities	<code>get_joint_velocities()</code>
command gripper	<code>send_gripper(...)</code> (→ Fairino MoveGripper)
read gripper width / force / position	— none —

There is no `GetGripperStatus` / width / force read in the wrapper (or surfaced from the Fairino SDK here). So at deploy time the policy (and `deploy.py`) literally cannot tell *closed-on-object* from *closed-on-air*. Any gate written today would have nothing to check.

What signal is needed (pick one)

Signal	How	Notes
Gripper width / position read	a Fairino SDK call (if the firmware/gripper supports it) wrapped in <code>FR5Controller</code>	best option — directly distinguishes a closed-on-object (jaws stop early) from closed-on-air (jaws fully close).
Gripper force / current	force-controlled gripper reporting contact force / motor current	a force spike at contact = a real grasp.
Pseudo-tactile (finger angle)	infer from the force-gripper's own joint angle (no extra hardware) — cf. arXiv:2503.23835	the cited paper's approach; codeable if the gripper exposes its angle.
Wrist-camera object-in-jaw detector	a small detector on the wrist frame after the close	no new hardware, but needs a (heuristic or learned) detector and the object visible between the jaws.
Wrist F/T sensor	a force-torque sensor at the wrist	clean contact signal; needs the sensor.

Where it plugs in (once a signal exists)

`common/deploy.py`, in the control loop, **between the gripper command and the arm execution** (after `gripper.update(gripper_cmd)`, before `servo_j` / the delta-EEF IK step):

```
gripper.update(gripper_cmd)                # send open/close command
if just_commanded_close:
    grasped = robot.grasp_detected()        # <-- the missing signal
    (width/force/tactile/vision)
    if not grasped:
        print("[GRASP] mid-air close — re-approaching, not transporting")
        # hold / re-run the approach instead of proceeding to transport
        ... # e.g. lift 5 cm, re-open, retry the reach; do NOT advance the task phase
        continue
    # only reaches here (transport) when a grasp is confirmed (or it wasn't a close)
    robot.servo_j(joints_cmd) # or the delta-EEF IK path
```

Until `robot.grasp_detected()` (or equivalent) returns a real signal, the gate is a no-op and is intentionally left out. Add the wrapper method in `../so101-fr5-teleop/fr5.py` first, then this gate, then test that a mid-air close triggers a retry rather than a phantom transport.

Reference: *Disambiguate Gripper State in Grasp-Based Tasks* (arXiv:2503.23835) — the open-empty / closed-on-object / closed-on-air ambiguity and a pseudo-tactile fix.

8. The Fix — what we actually changed (branch `fix/dino-act-cvae-probe`)

Files 1–7 *diagnose* the DINOv2 ACT failure. This file documents **what we did about it** on the `fix/dino-act-cvae-probe` branch: every change, why, and how to verify it. It targets the two *fixable in code* root causes — the **CVAE posterior collapse** and the **lack of a vision-vs-state model-selection metric** — and ports the real DINOv2 model so the fixes land on the actual failing setup. Scope is **train + offline probe** (no deploy changes); the grasp gate is documented only (file 7), and gripper-dropout was intentionally dropped as over-engineering.

One-line: keep the CVAE latent **alive** so ACT stops mode-averaging, and add a probe that measures whether the policy uses the **cameras** vs its own **joint state** so you can pick models by that instead of by validation loss.

8.1 The CVAE KL fix (the headline change)

Problem (file 3 §3.2 / file 6): with `kl_weight=10` baked into lerobot’s loss, the KL term collapsed to ~ 0 in every run ($0.21 \rightarrow 0.0001$). A dead latent = ACT degenerates into a plain mean-regressor = the object-blind “average” grasp.

What we changed: `policies/act/model.py` `ACT.forward` no longer uses lerobot’s pre-combined loss. It replicates lerobot’s forward (the `OBS_IMAGES` stack + `self.model(batch)` call) but **recomputes the KL term** with three anti-collapse mechanisms. Plus new config keys.

```
# per-latent-dim KL (shape (B, 32))
kld_dim = -0.5 * (1 + log_sigma_x2 - mu.pow(2) - log_sigma_x2.exp())
# (2) FREE-BITS: floor each dim at lambda before summing
kld = kld_dim.clamp(min=self.kl_free_bits).sum(-1).mean()
# (1) ANNEALING: ramp the weight 0 -> kl_weight over kl_warmup_steps
kl_w = self.kl_weight * min(1.0, _kl_step / self.kl_warmup_steps)
loss = l1 + kl_w * kld # (3) lower kl_weight (10 -> 0.5)
```

Config (`config.yaml` + the dino configs): `kl_weight: 0.5`, `kl_warmup_steps: 2000`, `kl_free_bits: 0.03` (smoke/local use smaller warmups).

Honesty note (read this). The CVAE change is the **most-deviating, least-certain** part of this branch — the ACT paper uses `kl_weight = 10` and *deliberately* keeps the latent **weak** (because $z = 0$ at inference). The well-evidenced root cause is the **proprioceptive shortcut** (the measured $2.2\times$ state>vision), not the CVAE. So the free-bits floor was deliberately set **gentle** ($0.03/\text{dim} \approx 1 \text{ nat}$, within the healthy 0.5–5 KL range) — just enough to stop KL hitting exactly zero, *not* enough to force a strong latent that would mismatch $z = 0$ at deploy. Treat the KL change as a small, **evidence-gated** tweak: watch the training KL curve and the probe, and if it doesn't help, set `kl_weight: 10` + remove free-bits to fall back to the proven recipe.

Why three mechanisms — and what each one is

The KL term measures how much information the latent z carries. Reconstruction wants z to carry **lots** (helps predict the action); the KL wants it to carry **none** (stay at the prior). At `kl_weight=10` the KL side won outright → collapse. The three fixes each weaken that pull:

(1) KL annealing — “ease into the regularization.” Don't apply the KL penalty from step 1; start it at **0** and ramp to the target over `kl_warmup_steps` (2000). *Why:* at the start, z is useless noise (the decoder hasn't learned to use it), so full KL pressure makes the optimizer take the cheap path — zero z out — and then the decoder never learns to use it (chicken-and-egg → permanent collapse). With the weight ≈ 0 early, the model is **free to learn to use z** first (reconstruction makes it informative); *then* the KL is gently turned on so z stays well-behaved but alive. - At step 0: `kl_w = 0` → loss = pure reconstruction (verified: `loss == l1` at step 0). - At step ≥ 2000 : `kl_w = 0.5` (full target). - `_kl_step` is a saved buffer that advances on each training step (resume-safe). - *Analogy:* teach someone to **use** a tool before grading them on a tidy workbench.

(2) Free-bits — “a tax-free allowance per latent dimension.” Give each of the 32 latent dims a small amount of KL it may use **for free** (no penalty). Floor the per-dim KL at $\lambda = 0.1$. Below the floor the optimizer gets **no reward** for shrinking further, so it can't squeeze a dim to zero — each dim keeps $\geq \lambda$ nats of information. - `kld_dim.clamp(min=0.1)` replaces any sub-0.1 dim with a flat `0.1`; a flat value has **zero gradient**, so there is no force pushing it lower. The penalty only “bites” on info **above** 0.1. - *Analogy:* a tax-free allowance — each dim carries 0.1 nats tax-free; only info above that is taxed.

(3) Lower weight. `kl_weight: 10 → 0.5` — turn the overall KL pull down so the “use z ” force isn't overwhelmed. `10` was the single biggest collapse trigger.

Why a non-zero KL fixes the object-blind grasp

A **live** latent means ACT can put “which demonstration mode this is” into z and, at inference ($z = 0$), **commit to one mode** instead of averaging all of them. Averaging is exactly what produced the phantom “mean” reach + mid-air grasp (file 2 §2.3). Keep z alive → ACT can pick a mode → less object-blind averaging.

How to verify (Linux/GPU box)

- Every run writes a per-epoch `<checkpoint_dir>/metrics.csv` with the columns `train_l1`, `train_kl`, `val_l1`, `val_kl`, `train_val_gap`, `grad_norm`, `kl_weight_eff`, `lr`, `seconds`. Plot it: `python tools/plot_metrics.py <checkpoint_dir>` — it saves a PNG and prints a verdict. The KL should **level off above zero** (~0.5–5), not crash `0.21 → 0.0001`. (The plotter flags `final KL >= 0.1 → HEALTHY` vs `→ COLLAPSED`.)
- Feed the **same** image with two different `z` samples → the predicted action chunk should **visibly differ** (proof `z` is being used).

Logging captured for error analysis (next runs)

Every run also drops a one-time `<checkpoint_dir>/run_info.json` provenance snapshot: timestamp, **git commit/branch/dirty**, exact resolved config, dataset root + `action_space` + train/val frame counts + `frame_stride` + `image_size`, device, torch/python/platform, and the trainable-param count. This ties any later deploy failure back to *exactly* the code, config, data split, and environment that produced the checkpoint. The extra `metrics.csv` columns are the in-training diagnostics: `val_kl` (rising while `train_kl` stays low ⇒ the latent is overfitting), `grad_norm` (a sudden spike precedes divergence), and `train_val_gap` (the overfitting signal — file 9). Together with the existing checkpoint payload (`config`, `stats`, `action_space`, `val_l1`) this is everything needed to error-analyse a run after the fact.

8.2 The DINOv2 2-camera ACT port

So the fixes apply to the *real* failing model (not the base ResNet ACT). Ported from vivek's `act` branch, **without gripper-dropout** and merged carefully with this repo's delta-EEF work.

File	Change
policies/act/model.py	DinoV2Backbone (frozen DINOv2 ViT-S/14, optional last-N trainable blocks) swapped in for lerobot's ResNet18; multi-camera _make_batch that accepts a single tensor or a {camera_key: tensor} dict; predict_chunk() for the probe.
common/dataset.py	per-camera image keys (observation.images.<cam>), frame_stride , and a photometric aug level.
common/convert_episodes.py	wrist+scene extraction + --extract-stride (kept the delta-EEF / action_space path).
common/train.py	passes a single image tensor when there's 1 camera (so the other 5 policies are untouched) and a dict when there are 2 (ACT only); the DINOv2 backbone trains at 0.1× LR .
policies/act/config.joint_dino{,_ft,_prop}.yaml	2-camera DINOv2 configs (frozen / last-block-FT / proprio-dropout 0.5), all with the CVAE fix, no gripper-dropout.

Key design choice (lean & non-breaking): the multi-camera convention only activates with 2+ cameras. The other policies always train on single-camera data, so they receive a single tensor exactly as before — verified unchanged by tools/test_inference.py (all pass) and the diffusion smoke test.

Upgraded vision encoder — DINOv2 / DINOv3 / I-JEPA (configurable)

DinoV2Backbone was generalised to **VisionBackbone**, which loads any of three frozen self-supervised ViTs and reshapes their patch tokens to lerobot's (B, C, h, w) feature map (it drops any leading CLS/register tokens automatically, so the same code handles all three):

dino_backbone	Loader	Size / patch	Notes
dinov2_vits14	torch.hub	21M / p14	the original (open, no auth)
dinov3_vits16 (new default for the dino configs)	HF AutoModel	21M / p16	upgrade — stronger features, same footprint. GATED: <code>huggingface-cli login</code> with a token that accepted Meta's license.
dinov3_vitb16 / dinov3_vitl16	HF AutoModel	86M / 300M	bigger DINOv3
ijepa_vith14	HF AutoModel	~630M	open, but GPU-only (smallest public I-JEPA is ViT-H)
any owner/model	HF AutoModel	—	load any HF ViT id verbatim

The internal net is kept as `self.dino` so `train.py`'s `backbone.dino` 0.1x-LR match still works. The three `config.joint_dino*.yaml` now default to `dinov3_vits16`; set `dino_backbone: dinov2_vits14` to revert (no auth). Verified by a mock test of the patch-extraction/reshape for all three families (no weights downloaded locally); the actual DINOv3/I-JEPA load happens on the Linux/GPU box (DINOv3 needs the HF login; I-JEPA needs the GPU).

8.3 The vision-vs-state probe + model selection

Problem (file 3 §3.3): checkpoints were picked by `val_l1`, which is *anti-correlated* with deploy success (the deployed model had the **worst** val loss).

What we added: `experiments/shortcut_probe.py`. For each validation sample it takes the predicted action chunk (`predict_chunk`, bypassing temporal ensembling) under input perturbations and measures how much it MOVES:

- **zero-images** → `image_sensitivity` (does blanking the cameras change the action?)
- **shuffle-images** (`state_A` + another sample's images = the conflict-swap) → `image_content_sensitivity`
- **zero-state** → `state_sensitivity`
- **output_spread** ($\approx 0 \Rightarrow$ a canned/constant trajectory) and `action_std` (the GT scale)

Headline metric: `state / image ratio = state_sensitivity / image_sensitivity`. `>> 1` = object-blind (the deployed model was **~2.2x**); `< 1` = image-dominant (good).

Model selection: `--checkpoints a.pt b.pt ...` probes each, **ranks by vision-dominance**, and recommends the most image-dominant — the metric to use **instead of** `val_l1`.

```
python experiments/shortcut_probe.py --ckpt
    policies/act/checkpoints_joint_dino/best.pt
python experiments/shortcut_probe.py --checkpoints run_a/best.pt run_b/best.pt
    run_c/best.pt
```

8.4 The grasp-success gate — documented, not coded

The mid-air-grasp fix needs a real grasp signal, but the FR5 gripper is command-only (no width/force/position read). So it is **deferred** with a precise spec of the required signal and the `deploy.py` insertion point — see [07_grasp_gate_requirements.md](#).

8.5 Full change set (21 files, +2,106 lines)

code:	policies/act/model.py	DIN0v2 + multi-cam + predict_chunk + CVAE
fix	common/dataset.py	per-camera keys + frame_stride +
photometric	common/convert_episodes.py	wrist+scene extraction + --extract-stride
	common/train.py	single-or-dict image passing + 0.1x DIN0v2
LR	common/smoke_test.py	new per-camera key assertions
	experiments/shortcut_probe.py	vision-vs-state probe + model selection
configs:	policies/act/config.yaml / .local / .smoke	kl_weight 10→0.5 + warmup
+ free-bits	policies/act/config_joint_dino{,_ft,_prop}.yaml	2-cam DIN0v2 configs
docs:	docs/failure_analysis/01-08	the analysis + this fix log + grasp-gate
spec		

Unchanged (no regressions): the other 5 policies (diffusion, dit_flow, pi0/pi05/pi0_fast), `deploy.py`, `common/proprio.py`.

8.6 What's verified vs what's left

Verified on CPU (Mac): ACT 1-cam & 2-cam build/forward/backward/predict/predict_chunk; KL annealing (`loss == l1` at step 0) + free-bits + `_kl_step` increment; smoke tests (act, diffusion) end-to-end; `test_inference` all-pass; the probe runs end-to-end. DIN0v2's `torch.hub` load itself was **not** triggered locally (avoided the download).

Left for the Linux/GPU box: 1. Train a few epochs on a dino config → confirm **KL stabilizes > 0** (not `0.21 → 0.0001`). 2. `shortcut_probe.py --ckpt <best>` → confirm the model is **less state-dominant than 2.2x**; use `--checkpoints ...` to select. 3. Point `dataset.root` at the 2-camera dataset (re-convert with `--cameras wrist,scene --extract-stride 5` if needed).

Still the real-world root fix (out of this branch's scope): re-record **fixed-home + varied marker** data — no code change fully removes the proprioceptive shortcut if the data still lets the joint state predict the grasp (file 4 §4.2).

9. “Is the model too small?” — No. It’s overfitting, not under-capacity.

A natural question, given the architecture *looks* modest (a 32-dim CVAE latent, `d_model=512`) and the **train/val loss gap is large** (train ~0.05, val ~0.14–0.17 — see file 3): *do we just need a bigger model?*

Short answer: no. A large train/val gap is the textbook signature of **overfitting**, which means the model has **enough (if anything, too much) capacity for the data** — not too little. Growing the model would most likely make the gap **worse**.

9.1 Under-capacity vs overfitting are opposite signatures

	train loss	val loss	gap	what it means	fix
Underfitting (too small / under-trained)	high	high	small	model can’t even fit the training data	more capacity / longer training
Overfitting (our case)	low (~0.05)	high (~0.15)	large (~0.10)	model <i>memorizes</i> the train set, doesn’t generalize	more / more-diverse data , regularization, best-checkpoint selection

Our numbers (train ~0.05, val ~0.15, gap ~0.10) are squarely the **overfitting** row. If the model were too small we’d see *both* losses stuck high with a *small* gap — the opposite of what the logs show.

9.2 The 32-dim latent is standard — and isn’t the capacity knob anyway

`d_model=512`, `latent_dim=32`, 4 encoder / 7 decoder layers is the **canonical ACT configuration** — exactly what the ACT paper (ALOHA) and lerobot use, and what people run on custom bots. This repo is, if anything, slightly *larger* than lerobot’s default (7 decoder layers).

Crucially, `latent_dim=32` is the CVAE “**style**” latent — it encodes *which demonstration mode* this is, not the model’s main representational capacity. Bumping it does **not** add the kind of capacity that would close a generalization gap; it just gives the (already-collapsing, file 6) latent more dimensions to ignore.

9.3 The decisive evidence: it's data-bound, not capacity-bound

From the training-log analysis (file 3): the **94M-parameter paper-scale ACT** had the **same validation loss as the small one**. When *more capacity does not lower val loss*, the bottleneck is **data, not parameters** — adding params just memorizes the same small dataset faster. With ~54–86 episodes, the model is data-limited.

This matches community practice: ACT overfitting is universally treated as a *data / checkpoint* issue (“divergence between train and val curves — do **not** use the final checkpoint; pick the lowest-val one”), and the tuning order when success is low is **chunk size** → **β /KL** → **learning rate** — *never* “add parameters.”

9.4 What people actually do for production ACT on custom bots

- **Keep the standard 512/32 arch.** Nobody scales the ACT backbone for a single custom task.
 - **Scale data, not the model.** ~50 demos → 80%+ on a simple pick-place is typical; the lever is **more diverse demos** (100–150, varied object positions), not more params.
 - **Reduce trainable capacity to fight overfitting** — which is exactly what a **frozen DINOv2/DINOv3 backbone** does (it removes ~11M trainable ResNet params and replaces them with a strong, frozen visual prior the 54-episode set could never learn itself). Plus photometric augmentation and **best-checkpoint selection** (by the probe, file 8 / `experiments/shortcut_probe.py`, not `val_l1`).
 - **Bigger model only if** you move to *multi-task / many-object generalization* **and** bring a lot more data — and even then the right move is a **pretrained VLA** (the **Octo** finetune in this repo, or $\pi 0$), not a bumped `latent_dim`.
-

9.5 What actually closes the gap (in priority)

1. **More + more-diverse data** (fixed-home + varied-marker, 100–150 demos) — the real fix, and the one that also kills the proprioceptive shortcut (file 2 §2.2 / file 4 §4.2).
2. **Frozen pretrained backbone** (DINOv2/v3) — fewer trainable params + a strong visual prior.  done.
3. **Augmentation + checkpoint selection by the vision-vs-state probe** —  in place.
4. A **pretrained VLA** (Octo / $\pi 0$) if you need broad generalization — not a bigger ACT.

One-line takeaway: the 32-dim latent and 512-wide ACT are correct and standard; the large train/val gap is **overfitting on a small dataset**, so the cure is **more/diverse data + the frozen pretrained encoder + probe-based checkpoint selection** — *not* more parameters. Growing the model would likely widen the gap.

References

- Zhao et al. — *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware* (ACT) — RSS 2023 — [arXiv:2304.13705](https://arxiv.org/abs/2304.13705)
- lerobot ACT config — github.com/huggingface/lerobot
- ACT practical guide (SVRC) — <https://www.roboticscenter.ai/blog/act-policy-explained>
- Shaka-Labs ACT (sub-30-demo training) — <https://github.com/Shaka-Labs/ACT>
- See also file 3 (`03_training_log_findings.md` — the val-plateau / capacity finding) and file 4 (`04_root_cause_and_fixes.md` — the data fix).

10. Free-bits, explained in detail (the “tax-free allowance” for the latent)

This is a focused, slow walkthrough of **free-bits** — one of the three CVAE anti-collapse mechanisms (file 8 §8.1). The name is unhelpful; the idea is simple once you see it as a *tax*. For the broader KL/CVAE context read [06_cvae_kl_deep_dive.md](#) first; this file zooms all the way in on free-bits.

10.1 First: what the KL term does to the latent

The CVAE loss is $L = \text{reconstruction}(L1) + \text{kl_weight} \cdot \text{KL}$. The **KL term is a penalty** in the loss. It penalizes the latent z for carrying information. Because the optimizer *minimizes* the loss, it tries to make the KL **small** — which means it pushes the information in z toward **zero**.

The total KL is a sum over all 32 latent dimensions:

$$\text{KL_total} = \text{kl_1} + \text{kl_2} + \dots + \text{kl_32}$$

where each kl_i measures *how much information dimension i carries* (in **nats**, a unit of information). $\text{kl_i} = 0$ means dimension i is **dead** — it carries nothing and the decoder can’t use it.

The collapse: because the optimizer wants the KL small, it drives **every** $\text{kl_i} \rightarrow 0$. All 32 dims die $\rightarrow$ the latent carries nothing $\rightarrow$ the decoder learns to ignore $z \rightarrow$ ACT degenerates into a plain mean-regressor (mode averaging $\rightarrow$ the object-blind grasp). That is exactly what the FR5 training logs showed (KL $0.21 \rightarrow 0.0001$).

10.2 The mental model: KL is a *tax on information*

Imagine that every nat of information you store in the latent gets **taxed** by the KL penalty. What does a rational tax-avoider do? Store **nothing**. Empty latent $\rightarrow$ collapse.

That is the whole problem in one image: **the KL taxes information, so the model avoids storing any**.

10.3 Free-bits = a tax-free allowance per dimension

Free-bits changes the rule: **the first λ nats in each dimension are tax-free**. Only information *above* λ gets taxed.

Formally, instead of penalizing kl_i , you penalize $\max(kl_i, \lambda)$:

$$KL_{\text{used}} = \max(kl_1, \lambda) + \max(kl_2, \lambda) + \dots + \max(kl_{32}, \lambda)$$

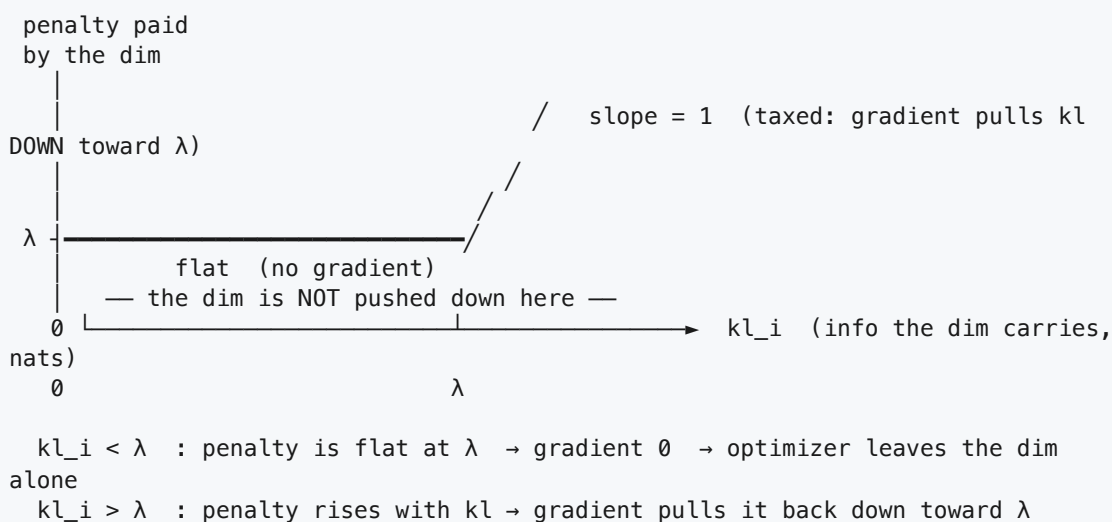
- If a dimension carries **more** than $\lambda \rightarrow \max(kl_i, \lambda) = kl_i \rightarrow$ it is taxed, and the KL still pulls it down toward λ .
- If a dimension carries **less** than $\lambda \rightarrow \max(kl_i, \lambda) = \lambda$, a **constant** $\rightarrow$ its derivative is **zero** $\rightarrow$ there is **no gradient pushing it any lower**.

So the optimizer **gains nothing** by squeezing a dimension below λ . Below the floor the KL stops fighting, and the **reconstruction** loss is then free to use that dimension (up to λ nats) because it costs nothing. Net effect: each dimension keeps $\sim \lambda$ nats of usable information, and the latent **stays alive**.

One sentence: **free-bits gives each latent dimension a tax-free allowance of λ nats — the model can store that much “for free,” so it stops emptying the latent to zero.**

10.4 The gradient view (why “no push below the floor”)

The reason free-bits works is entirely about the **gradient** of $\max(kl, \lambda)$. Plot the penalty a dimension pays as a function of how much info it carries:



- **Right of λ** (a dim carrying a lot): the KL tax has a downward gradient $\rightarrow$ it shrinks the dim back toward λ . (Normal regularization — keeps the latent from carrying *too* much, which would break $z = 0$ inference.)
- **Left of λ** (a dim nearly dead): the penalty is a flat line $\rightarrow$ **gradient is zero** $\rightarrow$ the KL does **not** drag it to zero. The dimension is “parked” with up to λ nats it may use for free.

Without free-bits the curve is just $\text{penalty} = kl$ (a straight line through the origin with slope 1 everywhere) $\rightarrow$ there is *always* a downward gradient $\rightarrow$ every dim is dragged to 0 $\rightarrow$ collapse. Free- bits flattens the curve below λ , removing that downward pull exactly where

collapse happens.

10.5 A tiny worked example ($\lambda = 0.5$, 4 dimensions)

Dimension's kl_i	Without free-bits (penalty = kl_i)	With free-bits (penalty = $\max(kl_i, 0.5)$)
0.1 (nearly dead)	0.1 — gradient pulls it to 0 ❌ dead	0.5 (flat) — gradient 0 → NOT pushed down ✅ kept
0.5 (at the floor)	0.5 — pulled toward 0	0.5 — no downward pressure
2.0 (carries a lot)	2.0 — pulled down hard	2.0 — taxed only on the part above 0.5

Without free-bits, that first dimension gets dragged to 0 and dies. With free-bits it keeps its 0.5 nats for free, so the model actually **uses** it instead of abandoning it.

10.6 Why it *specifically* stops the collapse (the chicken-and-egg)

Posterior collapse is a vicious cycle:

```
early in training: z is random noise → it doesn't help reconstruction yet
"defend" it → reconstruction has no reason to
kills it:  $kl_i \rightarrow 0$  → the KL tax (downward gradient)
never learns to use it → now the dim is dead, the decoder
→ permanently collapsed
```

Free-bits breaks the cycle at the third arrow: **below λ there is no downward KL gradient**, so the dims are *not* forced to zero. They stay parked with a small free budget, which gives reconstruction the chance to start using them once it learns how. (KL **annealing** — file 8 — helps the same cycle from a different angle: it removes the KL pressure *entirely* early on so reconstruction can learn to use z first; free-bits is the *steady-state* guarantee that no dim hits exactly zero.)

10.7 In our code (and the exact value we use)

policies/act/model.py, in ACT.forward:

```
# per-latent-dim KL: kld_dim has shape (B, 32)
kld_dim = -0.5 * (1 + log_sigma_x2 - mu.pow(2) - log_sigma_x2.exp())
# FREE-BITS: floor each dim at lambda, THEN sum over dims, THEN mean over batch
kld = kld_dim.clamp(min=self.kl_free_bits).sum(-1).mean()
#           ↳ clamp(min=λ) is exactly max(kl_i, λ) per dimension ↳
```

- `clamp(min=0.03)` replaces any dimension whose KL is below **0.03** with a flat `0.03` → no gradient below it → that dimension cannot be squeezed to zero.
- With 32 dims, the latent keeps **at least** $\sim 0.03 \times 32 \approx 1 \text{ nat}$ of information, instead of collapsing to ~ 0 .

Why $\lambda = 0.03$ (gentle), not 0.1

We deliberately use a **small** floor. ACT *intends* the latent to be a **weak** “style hint” (because $z = 0$ at inference — see file 8 §8.1). A large floor (the original $0.1/\text{dim} \approx 3.2 \text{ nats}$) would *force* the latent to carry a lot, then zero it at deploy → a train/test mismatch. $0.03/\text{dim}$ (~ 1 nat total, inside the healthy 0.5–5 KL band) is just enough to stop the latent dying, **not** enough to over-force it. The free-bits floor is intentionally minimal; it is the cheap insurance, not the main lever (the lower `kl_weight` is).

10.8 One-paragraph summary

The KL term taxes the latent for carrying information, so the optimizer empties the latent to zero (posterior collapse). Free-bits gives each of the 32 latent dimensions a **tax-free allowance of λ nats** — implemented as `max(kl_i,  $\lambda$ )` (in code, `clamp(min= $\lambda$ )`): above λ the dimension is taxed normally (gradient pulls it down toward λ), but **below λ the penalty is flat so its gradient is zero**, meaning the optimizer never drags it to zero. This keeps every dimension carrying at least λ nats of usable information, breaks the collapse, and — kept deliberately small ($\lambda = 0.03/\text{dim}$) — does so without forcing the latent to be strong enough to hurt the $z = 0$ inference that ACT relies on.

See also [06_cvae_kl_deep_dive.md](#) (KL/ELBO/reparameterisation from scratch) and [08_the_fix_implementation.md](#) §8.1 (free-bits + annealing + lower-weight together, and the honesty note).